We've just seen `ini_set()` above and how it can be used to retrieve directive values during execution. This function has a counterpart: the built-in functions `ini_get()` and `ini_set()` respectively allow you to determine or define directive values during run-time:

```
$ php -a
Interactive shell

php > var_dump(ini_get('memory_limit'));
string(2) "-1"
php > ini_set('memory_limit', '128M');
php > var_dump(ini_get('memory_limit'));
string(4) "128M"
```

Finally, you can also pass directives and values when PHP is invoked by using the `-d` switch. This functions in much the same way as environment variable manipulation:

```
$ php -r "var_dump(ini_get('memory_limit'));"
string(2) "-1"
$ php -d memory_limit=512M -r "var_dump(ini_get('memory_limit'));"
string(4) "512M"
```

Here we're executing code immediately with the `-r` command line switch and using `-d memory_limit=128M` to change the maximum memory PHP is allowed to allocate (a handy trick if you have certain scripts that need extra memory and you might not have ownership of the overall system configuration).

Module (Extension) Configuration

A lot of "core" PHP is actually implemented as extensions (also known as modules). You might remember in the Docker section earlier we ran the command `php -m`, and the output was a list of installed modules. Modules often have their own configuration files, which are typically located in a sub-directory of wherever your main PHP config is found – e.g., `/etc/php/current/mods.available` for Debian-based systems. At the very least, these files contain a line such as `extension=module_name.so`, which is an instruction to link a shared object resource to the main PHP executable when it runs.